

## 说明

本应用笔记介绍了基于 CH32 的 USB-DFU 固件升级实现方法，仅通过 USB 控制传输即可将固件下载到设备，无需烧录器或额外引脚。片内 Flash 划分为 Bootloader 与应用程序两个分区，配合通用主机工具 dfu-util，即可实现应用程序固件的下载与回读。

### 适用范围

| 适用范围   | 系列                       |
|--------|--------------------------|
| 通用 MCU | CH32L103 等支持 USBFS 的 MCU |

# 目录

|                           |    |
|---------------------------|----|
| 说明                        | 1  |
| 目录                        | 2  |
| 表格索引                      | 2  |
| 图片索引                      | 2  |
| 第 1 章 DFU 协议概述            | 1  |
| 1.1 DFU 类请求               | 1  |
| 1.2 DFU 状态机               | 1  |
| 1.3 主要数据帧                 | 1  |
| 1.3.1 GET STATUS 帧        | 1  |
| 1.3.2 DOWNLOAD FIRMWARE 帧 | 2  |
| 1.3.3 UPLOAD FIRMWARE 帧   | 2  |
| 1.3.4 零长度 DNLOAD 帧        | 2  |
| 1.3.5 ABORT 帧             | 2  |
| 第 2 章 DFU 实现              | 4  |
| 2.1 总体方案                  | 4  |
| 2.2 USB 描述符设计             | 4  |
| 2.3 软件实现                  | 4  |
| 2.3.1 软件架构                | 4  |
| 2.4 软件流程                  | 5  |
| 2.4.1 DFU 状态机与延迟操作机制      | 5  |
| 2.4.2 主机发送固件流程            | 6  |
| 2.4.3 擦除和设置地址流程           | 7  |
| 2.4.4 固件写入和读取流程           | 9  |
| 第 3 章 主机工具与升级操作           | 13 |
| 3.1 驱动安装                  | 13 |
| 3.2 主机工具                  | 14 |
| 历史版本                      | 16 |
| 声明                        | 17 |

# 表格索引

|                   |    |
|-------------------|----|
| DFU 类请求定义         | 1  |
| DFU 状态定义          | 1  |
| GET STATUS 响应格式   | 2  |
| 命令帧格式             | 2  |
| 零长度 DNLOAD 帧格式    | 2  |
| ABORT 帧格式         | 3  |
| 设备描述符关键参数         | 4  |
| DFU 功能描述符字段       | 4  |
| Bootloader 工程文件组成 | 5  |
| 主循环操作类型           | 5  |
| dfu-util 常用命令     | 14 |

# 图片索引

总体结构图 ..... 4

主机执行流程 ..... 6

擦除和设置地址抓包 ..... 7

擦除流程..... 7

固件写入抓包 ..... 9

固件写入流程 ..... 9

固件读取抓包 ..... 11

固件读取流程 ..... 11

未安装驱动设备状态 ..... 13

驱动安装完成后设备状态 ..... 13

列举 DFU 设备演示 ..... 14

下载固件演示 ..... 14

读取固件演示 ..... 15

第1章 DFU 协议概述

1.1 DFU 类请求

DFU 类请求通过控制传输的 SETUP 阶段下发，请求类型为 0x21(OUT 方向)或 0xA1(IN 方向)，wIndex 携带接口号。各请求定义如表 1-1 所示。

表 1-1DFU 类请求定义

| bRequest | 名称            | 数据方向  | wValue    | 功能说明                |
|----------|---------------|-------|-----------|---------------------|
| 0x00     | DFU_DETACH    | 主机→设备 | 超时时间      | 请求设备脱离当前接口          |
| 0x01     | DFU_DNLOAD    | 主机→设备 | wBlockNum | 向设备下载数据块，零长度表示传输结束  |
| 0x02     | DFU_UPLOAD    | 设备→主机 | wBlockNum | 从设备回读数据块            |
| 0x03     | DFU_GETSTATUS | 设备→主机 | 0         | 查询状态，返回 6 字节状态结构    |
| 0x04     | DFU_CLRSTATUS | 主机→设备 | 0         | 清除错误状态              |
| 0x05     | DFU_GETSTATE  | 设备→主机 | 0         | 查询状态码，返回 1 字节，不改变状态 |
| 0x06     | DFU_ABORT     | 主机→设备 | 0         | 中止当前操作并返回 dfuIDLE   |

1.2 DFU 状态机

协议共定义 11 个状态，主机与设备依靠 GETSTATUS/GETSTATE 同步状态。常用状态及含义如表 1-2 所示。

表 1-2DFU 状态定义

| 状态码 | 状态名称                   | 含义说明                          |
|-----|------------------------|-------------------------------|
| 0/1 | appIDLE/appDETACH      | 运行态空闲/等待 USB 复位               |
| 2   | dfuIDLE                | 升级态空闲，可接受 DNLOAD、UPLOAD       |
| 3   | dfuDNLOAD-SYNC         | 已收到 DNLOAD 请求，等待数据接收完成        |
| 4   | dfuDNBUSY              | 正在执行擦除/编程，主机应等待 bwPollTimeout |
| 5   | dfuDNLOAD-IDLE         | 一个数据块处理完成，可继续下一块              |
| 6   | dfuMANIFEST-SYNC       | 收到零长度 DNLOAD，传输结束进入固化阶段       |
| 7   | dfuMANIFEST            | 固化处理中                         |
| 8   | dfuMANIFEST-WAIT-RESET | 等待复位/重新枚举                     |
| 9   | dfuUPLOAD-IDLE         | 回读态，可继续 UPLOAD                |
| 10  | dfuERROR               | 错误态，需 CLRSTATUS 清除后恢复         |

1.3 主要数据帧

1.3.1 GET STATUS 帧

该帧的作用是轮询设备状态，当从机回复空闲时进行下一步数据传输，否则将会在设备回复的 bwPollTimeout 后再次轮询。设备响应主机的 GET STATUS 包的格式如表 1-3 所示。

表 1-3GET STATUS 响应格式

| 偏移  | 字段            | 长度 | 说明                                                                                                           |
|-----|---------------|----|--------------------------------------------------------------------------------------------------------------|
| 0   | bStatus       | 1  | 0x00=OK,0x03=errWRITE,0x04=errERASE,0x0E=errUNKNOWN                                                          |
| 1-3 | bwPollTimeout | 3  | 主机等待多久再次发起轮询，单位 ms                                                                                           |
| 4   | bState        | 1  | 0x02=dfuIDLE, 0x03=dfuDNLOAD_SYNC, 0x04=dfuDNBUSY, 0x05=dfuDNLOAD_IDLE, 0x06=dfuMANIFEST_SYNC, 0x0A=dfuERROR |
| 5   | iString       | 1  | 状态描述字符串索引(0x00)                                                                                              |

1.3.2 DOWNLOAD FIRMWARE 帧

DOWNLOAD FIRMWARE 的帧格式为[bmRequestType] [bRequest] [wValue] [wIndex] [wLength]其中 wValue 代表着 wBlockNum，wBlockNum<2 代表着特殊命令，wBlockNum≥2 代表着数据块索引，其中第一块数据块索引为 2。当 wBlockNum<2 时主机发送的为命令帧，当 wBlockNum≥2 时主机发送的为 1024 个字节的数据。命令帧格式如表 1-4 所示。

表 1-4 命令帧格式

| 偏移  | 长度 | 字段      | 含义说明                     |
|-----|----|---------|--------------------------|
| 0   | 1  | command | 0x41=擦除页命令，0x21=设置地址指针命令 |
| 1-4 | 4  | address | Flash 起始地址               |

1.3.3 UPLOAD FIRMWARE 帧

UPLOAD FIRMWARE 的帧格式为[bmRequestType] [bRequest] [wValue] [wIndex] [wLength]其中 wValue 代表着 wBlockNum，在该帧中 wBlockNum≥2，2 代表着主机想读取的第一个数据块。

1.3.4 零长度 DNLOAD 帧

该帧的含义为所有固件数据已发送完毕，进入 Manifest 阶段。零长度 DNLOAD 帧格式如表 1-5 所示。

表 1-5 零长度 DNLOAD 帧格式

| 偏移  | 长度 | 字段            | 含义说明                                |
|-----|----|---------------|-------------------------------------|
| 0   | 1  | bmRequestType | 0x21=OUT(0x00)   类(0x20)   接口(0x01) |
| 1   | 1  | bRequest      | DFU_DNLOAD                          |
| 2-3 | 2  | wValue        | 0                                   |
| 4-5 | 2  | wIndex        | DFU 接口编号                            |
| 6-7 | 2  | wLength       | 长度为 0                               |

1.3.5 ABORT 帧

该帧的含义为中止当前操作，bRequest 字段的值为 0x06。ABORT 帧格式如表 1-6 所示。

表 1-6ABORT 帧格式

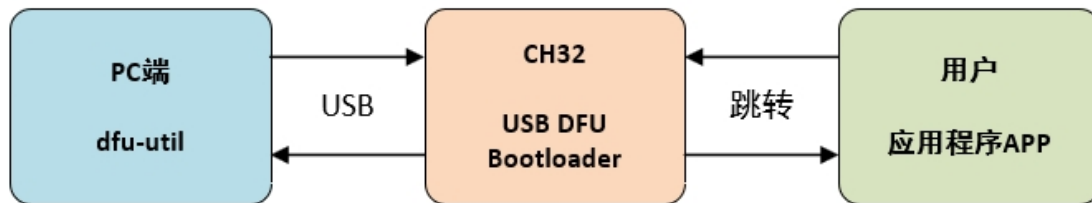
| 偏移  | 长度 | 字段            | 含义说明                                |
|-----|----|---------------|-------------------------------------|
| 0   | 1  | bmRequestType | 0x21=OUT(0x00)   类(0x20)   接口(0x01) |
| 1   | 1  | bRequest      | DFU_ABORT                           |
| 2-3 | 2  | wValue        | 0                                   |
| 4-5 | 2  | wIndex        | DFU 接口编号                            |
| 6-7 | 2  | wLength       | 长度为 0                               |

## 第2章 DFU 实现

### 2.1 总体方案

升级系统由三部分组成：PC 端的升级工具 dfu-util、CH32 单片机片内 Flash 中的 DFU Bootloader 以及用户应用程序 APP。总体结构如图 2-1 所示。

图 2-1 总体结构图



### 2.2 USB 描述符设计

设备描述符关键参数如表 2-1 所示。

表 2-1 设备描述符关键参数

| 字段              | 取值           | 说明            |
|-----------------|--------------|---------------|
| bcdUSB          | 0x0110       | USB1.10, 全速设备 |
| bDeviceClass    | 0x00         | 类定义在接口级       |
| bMaxPacketSize0 | 64 字节        | 控制端点包长        |
| idVendor        | 0x1A86       | WCH 厂商 VID    |
| idProduct       | PID          | 无要求           |
| iProduct        | CH32 DFU Dev | 产品字符串         |

配置描述符由配置描述符、接口描述符与 DFU 功能描述符组成，总长 27 字节，接口不使用额外端点（bNumEndpoints=0）。DFU 功能描述符各字段如表 2-2 所示。

表 2-2DFU 功能描述符字段

| 字段             | 取值      | 说明                                          |
|----------------|---------|---------------------------------------------|
| bmAttributes   | 0x0B    | bitCanDnload   bitCanUpload   bitWillDetach |
| wDetachTimeOut | 255ms   | 脱离超时时间                                      |
| wTransferSize  | 1024 字节 | 单次 DNLOAD/UPLOAD 最大数据量                      |
| bcdDFUVersion  | 0x011A  | DFU1.1a                                     |

接口字符串（iInterface=4）承载存储器布局字符串：以@Internal Flash/0x08005000/044\*001 Kg 为例，声明从 0x08005000 起连续 44 个 1KB 扇区、属性“g”表示可读可写可擦。dfu-util 解析该字符串后即可自动完成分区擦除与下载。

### 2.3 软件实现

#### 2.3.1 软件架构

Bootloader 工程在 CH32 USB 标准工程基础上增加 DFU 相关文件，各文件职责如表 2-3 所示。

表 2-3Bootloader 工程文件组成

| 文件                         | 职责                                  |
|----------------------------|-------------------------------------|
| main.c                     | 主流程：系统初始化、APP 有效性检查、DFU 初始化与主循环     |
| usb_dfu.c/usb_dfu.h        | DFU 类请求处理、状态机、延迟操作、Flash 编程与 APP 跳转 |
| usb_desc.c/usb_desc.h      | 设备/配置/DFU 功能/字符串描述符                 |
| ch32l103_usbfs_device.c/.h | USBFS 设备栈：枚举、标准请求、EP0 中断分发          |

USB 设备栈在 EP0 中断中完成标准请求分发，类请求（请求类型 0x21/0xA1）则转交 DFU\_SetupRequest()处理；数据阶段按方向分别回调 DFU\_EP0\_OutHandler()（接收 DNLOAD 数据）与 DFU\_EP0\_InHandler()（发送 UPLOAD 数据及 GETSTATUS 应答）。主程序完成启动判定后进入主循环，调用 DFU\_Process()处理延迟执行的任务。

EP0 中断中的类请求分发代码如下：

```
if ( ( USBFS_SetupReqType & USB_REQ_TYP_MASK ) == USB_REQ_TYP_CLASS ){
    errflag = DFU_SetupRequest();
    if( errflag == 0xFF ){
        USBFSD->UEP0_TX_CTRL = USBFS_UEP_T_TOG | USBFS_UEP_T_RES_STALL;
        USBFSD->UEP0_RX_CTRL = USBFS_UEP_R_TOG | USBFS_UEP_R_RES_STALL;
    }
    break;
}
```

2.4 软件流程

2.4.1 DFU 状态机与延迟操作机制

Flash 擦写耗时远超 USB 事务时限，若在中断中直接编程会阻塞 EP0 应答。为此状态机采用“中断内准备、主循环执行”的两段式设计：USB 中断中的处理只完成参数校验并登记待执行操作（DFU\_OP\_WRITE/DFU\_OP\_ERASE/DFU\_OP\_SET\_ADDR），随后进入 dfuDNBUSY 并在应答中返回 bwPollTimeout；真正的擦写和写入由主循环 DFU\_Process()执行，完成后状态切换为 dfuDNLOAD-IDLE，主机再次 GETSTATUS 即得到该状态并继续下一块。主循环操作类型表如表 2-4 所示。

表 2-4 主循环操作类型

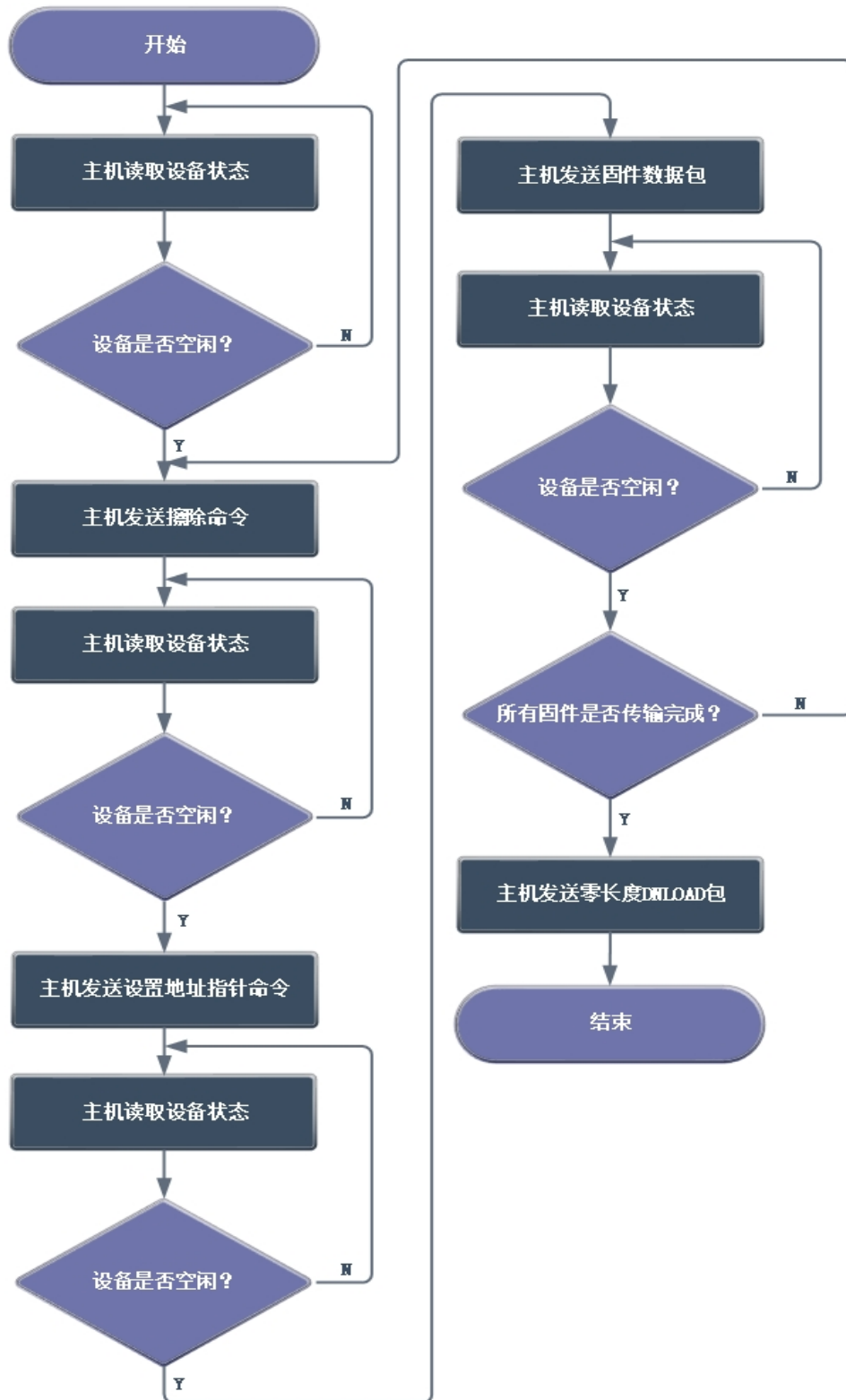
| 类型              | 值 |
|-----------------|---|
| DFU_OP_NONE     | 0 |
| DFU_OP_WRITE    | 1 |
| DFU_OP_ERASE    | 2 |
| DFU_OP_SET_ADDR | 3 |
| DFU_OP_JUMP_APP | 4 |



### 2.4.2 主机发送固件流程

升级流程启动后，主机轮询设备状态，待设备空闲后依次下发擦除命令、设置地址指针命令，每发送一条指令都循环轮询等待设备回到空闲状态，之后循环发送固件数据包并持续查询设备状态，直至全部固件传输完成，最后主机发送零长度 DNLOAD 包，结束整个升级流程。主机执行流程图如图 2-2 所示。

图 2-2 主机执行流程



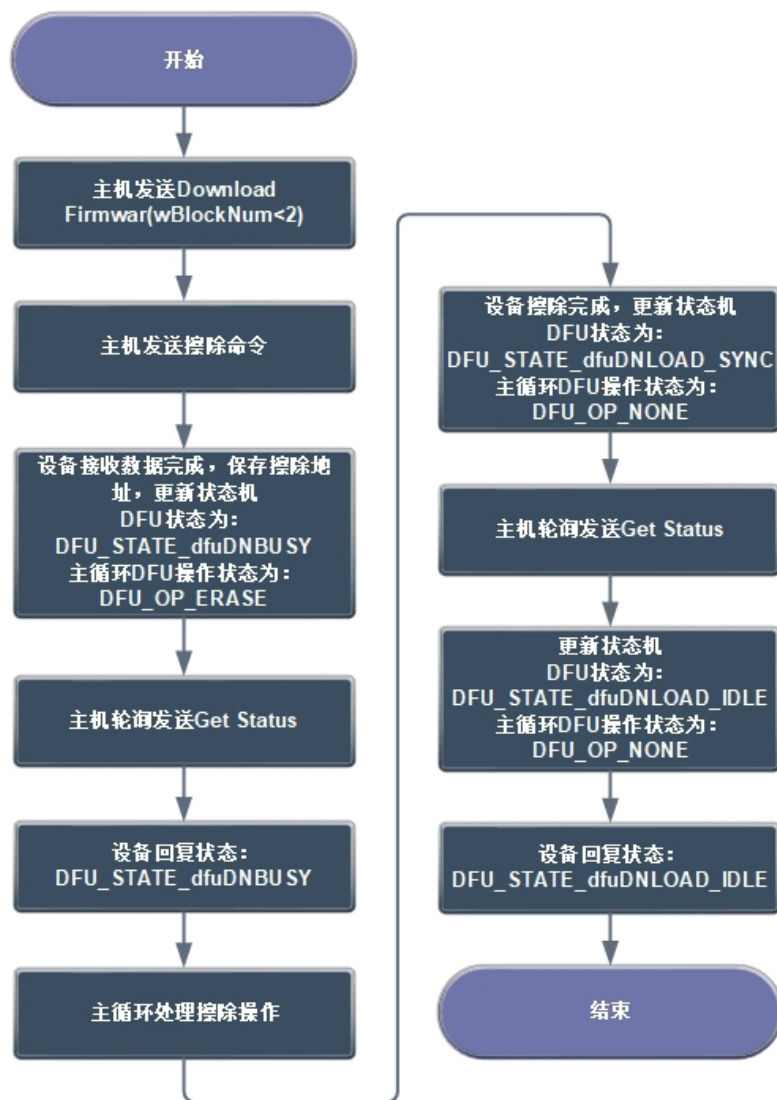
### 2.4.3 擦除和设置地址流程

主机下发 Download 固件块命令与擦除命令，设备接收后保存擦除地址并更新 DFU 状态机为忙状态、主循环标记为擦除操作，主机循环发送 Get Status 轮询查询设备状态，设备返回忙碌状态后由主循环执行实际擦除操作；擦除完成后设备更新 DFU 同步状态，主机继续轮询 Get Status，设备更新为空闲状态并回复空闲状态，流程结束，设置地址流程与该擦除流程逻辑类似，同样依靠主机下发命令、设备更新状态机、主机轮询状态、设备主循环执行对应操作完成。擦除和设置地址抓包图如图 2-3 所示，擦除流程图如图 2-4 所示。

图 2-3 擦除和设置地址抓包

|      |     |             |             |                   |            |
|------|-----|-------------|-------------|-------------------|------------|
| 61.0 | CTL | 21 01 00 00 | 00 00 05 00 | DOWNLOAD FIRMWARE | 204.1.0    |
| 61.0 | OUT | 41 00 50 00 | 08          | A.P.              | 204.2.0    |
| 61.0 | CTL | a1 03 00 00 | 00 00 06 00 | GET STATUS        | 205.1.0(2) |
| 61.0 | IN  | 00 0f 00 00 | 04 00       | .....             | 205.2.0    |
| 61.0 | CTL | a1 03 00 00 | 00 00 06 00 | GET STATUS        | 207.1.0    |
| 61.0 | IN  | 00 00 00 00 | 05 00       | .....             | 207.2.0    |
| 61.0 | CTL | 21 01 00 00 | 00 00 05 00 | DOWNLOAD FIRMWARE | 208.1.0    |
| 61.0 | OUT | 21 00 50 00 | 08          | I.P.              | 208.2.0    |
| 61.0 | CTL | a1 03 00 00 | 00 00 06 00 | GET STATUS        | 209.1.0    |
| 61.0 | IN  | 00 01 00 00 | 04 00       | .....             | 209.2.0    |
| 61.0 | CTL | a1 03 00 00 | 00 00 06 00 | GET STATUS        | 210.1.0    |
| 61.0 | IN  | 00 00 00 00 | 05 00       | .....             | 210.2.0    |

图 2-4 擦除流程



擦除与设置地址命令登记的相关代码如下：

```
static void DFU_PrepareDownload(void)
{
    uint32_t addr;
```

```

if (dfu_block_num == 0)
{
    uint8_t cmd = dfu_buffer[0];
    switch (cmd) {
        case DFU_CMD_SET_ADDRESS_POINTER:
            dfu_prog_addr = dfu_buffer[1]
                | ((uint32_t)dfu_buffer[2] << 8)
                | ((uint32_t)dfu_buffer[3] << 16)
                | ((uint32_t)dfu_buffer[4] << 24);
            dfu_pending_op = DFU_OP_SET_ADDR;
            dfu_poll_timeout = 1;
            dfu_state = DFU_STATE_dfuDNBUSY;
            break;

        case DFU_CMD_ERASE:
            addr = dfu_buffer[1]
                | ((uint32_t)dfu_buffer[2] << 8)
                | ((uint32_t)dfu_buffer[3] << 16)
                | ((uint32_t)dfu_buffer[4] << 24);
            if (addr >= DFU_APP_START_ADDR && addr <= DFU_FLASH_END_ADDR){
                dfu_pending_op = DFU_OP_ERASE;
                dfu_pending_addr = addr;
                dfu_poll_timeout = 15;
                dfu_state = DFU_STATE_dfuDNBUSY;
            }
            else{
                dfu_status = DFU_STATUS_errADDRESS;
                dfu_state = DFU_STATE_dfuERROR;
            }
            break;
    }
}
}

```

主循环处理擦除和设置地址的相关代码如下：

```

case DFU_OP_ERASE:
{
    uint8_t i;
    FLASH_Unlock_Fast();
    for (i = 0; i < 4; i++) {
        FLASH_ErasePage_Fast(dfu_pending_addr + i * DFU_FLASH_PAGE_SIZE);
    }
    FLASH_Lock_Fast();
    dfu_state = DFU_STATE_dfuDNLOAD_SYNC;
    break;
}

case DFU_OP_SET_ADDR:

```

```
dfu_state = DFU_STATE_dfuDNLOAD_SYNC;
break;
```

## 2.4.4 固件写入和读取流程

固件写入时，主机发送 Download 固件命令以及数据包，设备接收完成后更新状态机为忙碌状态，主循环标记为写入操作；主机持续轮询发送 Get Status 获取设备状态，设备返回忙状态，由主循环完成实际固件写入；写入完成后设备更新同步状态，主机继续轮询 Get Status，设备切换为空闲状态并回复空闲状态，完成本次固件数据包写入流程。固件写入抓包图如图 2-5 所示，固件写入流程图如图 2-6 所示。

图 2-5 固件写入抓包



图 2-6 固件写入流程



EP0 接收 DNLOAD 数据的相关代码如下：

```
void DFU_EP0_OutHandler(void)
{
    uint16_t rx_len;
    if (USBFS_SetupReqCode == DFU_DNLOAD){
        rx_len = USBFSD->RX_LEN;
        if (dfu_download_off + rx_len <= DFU_TRANSFER_SIZE){
            memcpy(&dfu_buffer[dfu_download_off], USBFS_EP0_Buf, rx_len);
            dfu_download_off += rx_len;
        }
        if ((rx_len < DEF_USBD_UEP0_SIZE) || (dfu_download_off >= dfu_download_len)){
            USBFSD->UEP0_TX_LEN = 0;
            USBFSD->UEP0_TX_CTRL = USBFS_UEP_T_TOG | USBFS_UEP_T_RES_ACK;
            dfu_data_ready = 1;
        }
        else{
            USBFSD->UEP0_RX_CTRL ^= USBFS_UEP_R_TOG;
        }
    }
}
```

固件数据块（wBlockNum≥2）登记的相关代码如下：

```
else if (dfu_block_num >= 2)
{
    uint32_t addr = dfu_prog_addr + (uint32_t)(dfu_block_num - 2) * DFU_TRANSFER_SIZE;
    if (addr < DFU_APP_START_ADDR || addr > DFU_FLASH_END_ADDR){
        dfu_status = DFU_STATUS_errADDRESS;
        dfu_state = DFU_STATE_dfuERROR;
        return;
    }
    dfu_pending_op = DFU_OP_WRITE;
    dfu_pending_addr = addr;
    dfu_pending_len = dfu_download_len;
    dfu_poll_timeout = 5;
    dfu_state = DFU_STATE_dfuDNBUSY;
}
```

主循环调用的写入执行的相关代码如下：

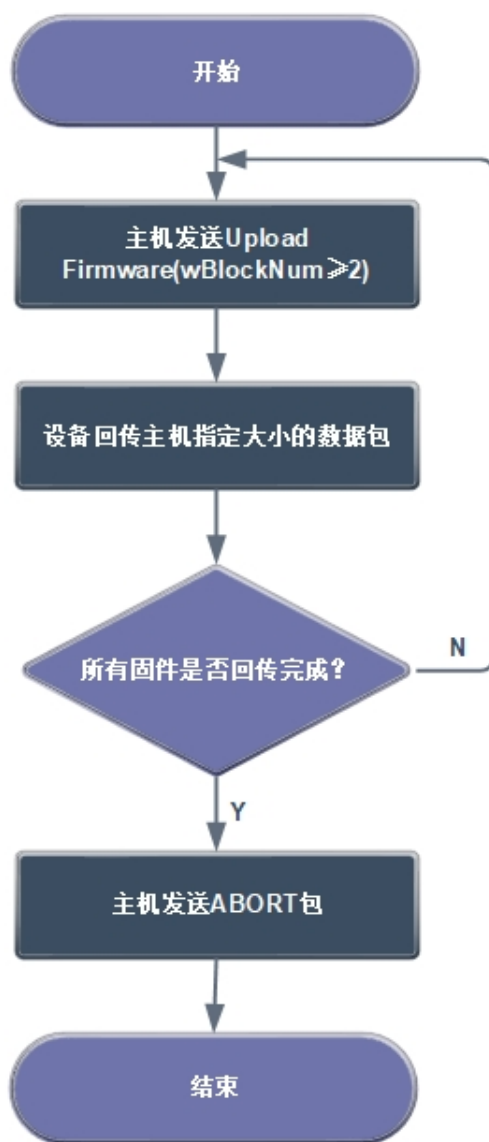
```
static void DFU_ExecuteDownload(void)
{
    switch (dfu_pending_op){
        case DFU_OP_WRITE:
            DFU_FlashProgram(dfu_pending_addr, dfu_buffer, dfu_pending_len);
            dfu_state = DFU_STATE_dfuDNLOAD_SYNC;
            break;
    }
    dfu_pending_op = DFU_OP_NONE;
    dfu_poll_timeout = 0;
}
```

与写入不同，固件读取的 Flash 读取操作耗时短，无需延迟执行机制，在 USB 中断内直接完成。固件读取的流程为主机发送 Upload 固件读取命令，设备向主机回传指定大小的数据包，判断固件是否全部回传完成，若未完成则循环继续执行上传操作，全部固件回传完毕后，主机发送 ABORT 包，中止固件上传流程。固件读取抓包图如图 2-7 所示，固件读取流程图如图 2-8 所示。

图 2-7 固件读取抓包

[illegible]

图 2-8 固件读取流程



UPLOAD 回读的数据源选择的相关代码如下：

```
if (dfu_block_num >= 2)
{
    uint32_t addr = dfu_prog_addr + (uint32_t)(dfu_block_num - 2) * DFU_TRANSFER_SIZE;
    if (addr > DFU_FLASH_END_ADDR){
        dfu_upload_src = 0; dfu_upload_avail = 0;
    }
    else{
        dfu_upload_src = (const uint8_t *)addr;
        uint32_t remaining = DFU_FLASH_END_ADDR - addr + 1;
        dfu_upload_avail = (remaining < DFU_TRANSFER_SIZE) ? (uint16_t)remaining : DFU_TRANSFER_SIZE;
    }
}
```

EP0 发送 UPLOAD 数据的相关代码如下：

```
void DFU_EP0_InHandler(void)
{
    uint16_t len;
    if (USBFS_SetupReqCode == DFU_UPLOAD){
        len = (USBFS_SetupReqLen > DEF_USBD_UEP0_SIZE) ? DEF_USBD_UEP0_SIZE : USBFS_SetupReqLen;
        if (len > 0 && dfu_upload_src != 0){
            memcpy(USBFS_EP0_Buf, dfu_upload_src, len);
            dfu_upload_src += len;
        }
        USBFS_SetupReqLen -= len;
        dfu_upload_off += len;
        USBFSD->UEP0_TX_LEN = len;
        USBFSD->UEP0_TX_CTRL ^= USBFS_UEP_T_TOG;
    }
}
```

## 第3章 主机工具与升级操作

### 3.1 驱动安装

DFU 设备需安装 WinUSB 驱动后方可正常使用。若未安装适配驱动，设备管理器中对应设备将显示黄色感叹号标识。当变更 VID 或 PID 后，需要为设备重新安装驱动才能正常使用；未安装驱动的 DFU 设备状态如图 3-1 所示，驱动安装完成后的设备状态如图 3-2 所示。

图 3-1 未安装驱动设备状态

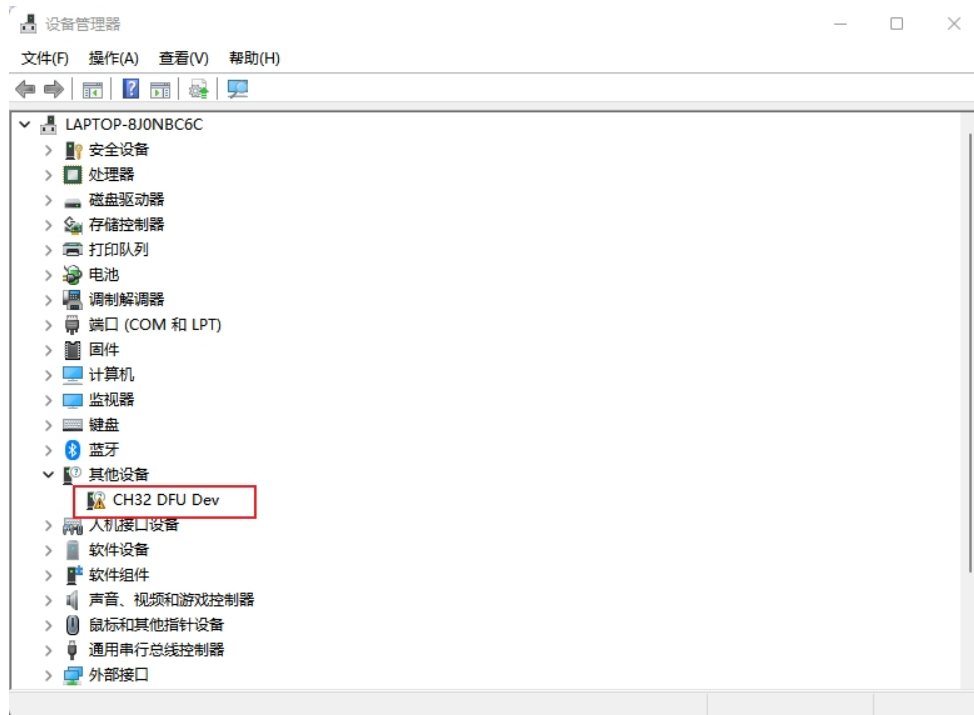
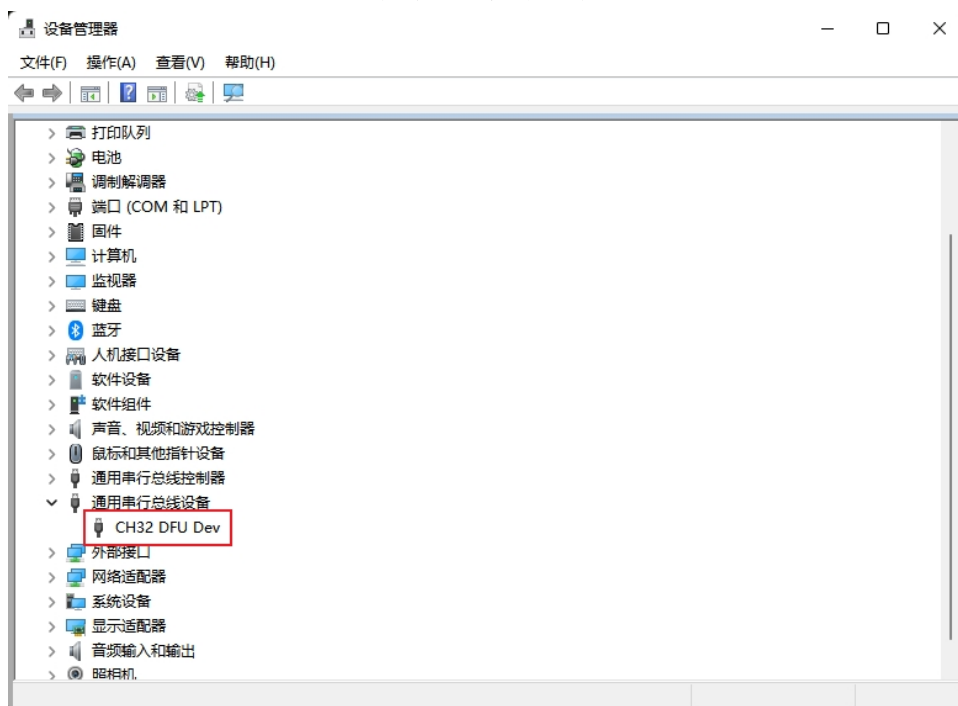


图 3-2 驱动安装完成后设备状态





### 3.2 主机工具

可直接使用开源命令行工具 **dfu-util** 进行固件写入和读取。dfu-util 常用命令如表 3-1 所示。

表 3-1dfu-util 常用命令

| 命令                                                                   | 功能                                           |
|----------------------------------------------------------------------|----------------------------------------------|
| dfu-util -l                                                          | 列举当前 DFU 设备                                  |
| .\dfu-util -d [VID]:[PID] -s [APP 程序的 FLASH 起始地址]:leave -D [APP.bin] | 指定特定的 VID 和 PID 设备从 APP 程序的 FLASH 起始地址开始下载固件 |
| .\dfu-util -d [VID]:[PID] -s [读取起始地址] -Z [读取字节数] -U [read.bin]       | 读取设备固件，将 Flash 内容回读到本地 bin 文件                |

在命令行中输入 dfu-util -l 可以列举出当前的 DFU 设备的相关信息。列举 DFU 设备演示图如图 3-3 所示。

图 3-3 列举 DFU 设备演示

```
Windows PowerShell
PS D:\下载\dfu-util-0.9-win64\dfu-util-0.9-win64> dfu-util -l
dfu-util 0.9

Copyright 2005-2009 Weston Schmidt, Harald Welte and OpenMoko Inc.
Copyright 2010-2016 Tormod Volden and Stefan Schmidt
This program is Free Software and has ABSOLUTELY NO WARRANTY
Please report bugs to http://sourceforge.net/p/dfu-util/tickets/

Cannot open DFU device 1a86:df11
Found DFU: [1a86:df11] ver=0100, devnum=31, cfg=1, intf=0, path="1-5.4", alt=0, name="@Internal Flash /0x08005000/044*001Kg", serial="DFU0001"
PS D:\下载\dfu-util-0.9-win64\dfu-util-0.9-win64> |
```

在命令行输入.\dfu-util -d 1a86:df11 -s 0x08005000:leave -D app.bin，即可将 app.bin 固件下载至设备 Flash，下载起始地址为 0x08005000。下载固件演示图如图 3-4 所示。

图 3-4 下载固件演示

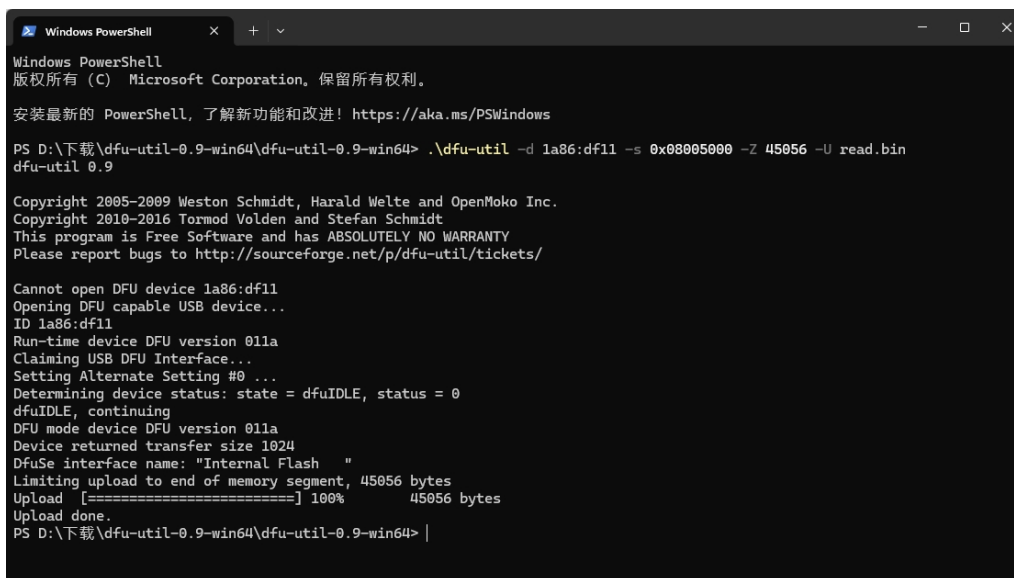
```
Windows PowerShell
Cannot open DFU device 1a86:df11
Found DFU: [1a86:df11] ver=0100, devnum=31, cfg=1, intf=0, path="1-5.4", alt=0, name="@Internal Flash /0x08005000/044*001Kg", serial="DFU0001"
PS D:\下载\dfu-util-0.9-win64\dfu-util-0.9-win64> .\dfu-util -d 1a86:df11 -s 0x08005000:leave -D app.bin
dfu-util 0.9

Copyright 2005-2009 Weston Schmidt, Harald Welte and OpenMoko Inc.
Copyright 2010-2016 Tormod Volden and Stefan Schmidt
This program is Free Software and has ABSOLUTELY NO WARRANTY
Please report bugs to http://sourceforge.net/p/dfu-util/tickets/

Invalid DFU suffix signature
A valid DFU suffix will be required in a future dfu-util release!!!
Cannot open DFU device 1a86:df11
Opening DFU capable USB device...
ID 1a86:df11
Run-time device DFU version 011a
Claiming USB DFU Interface...
Setting Alternate Setting #0 ...
Determining device status: state = dfuIDLE, status = 0
dfuIDLE, continuing
DFU mode device DFU version 011a
Device returned transfer size 1024
DfuSe interface name: "Internal Flash"
Downloading to address = 0x08005000, size = 16544
Download [=====] 100% 16544 bytes
Download done.
File downloaded successfully
Error during download get_status
PS D:\下载\dfu-util-0.9-win64\dfu-util-0.9-win64> |
```

在命令行中输入 `.\dfu-util -d 1a86:df11 -s 0x08005000 -Z 45056 -U read.bin` 可以从设备的 Flash 地址 0x08005000 开始读取 45056 字节固件至 read.bin 中。读取固件演示图如图 3-5 所示。

图 3-5 读取固件演示



```
Windows PowerShell
版权所有 (C) Microsoft Corporation. 保留所有权利。

安装最新的 PowerShell, 了解新功能和改进! https://aka.ms/PSWindows

PS D:\下载\dfu-util-0.9-win64\dfu-util-0.9-win64> .\dfu-util -d 1a86:df11 -s 0x08005000 -Z 45056 -U read.bin
dfu-util 0.9

Copyright 2005-2009 Weston Schmidt, Harald Welte and OpenMoko Inc.
Copyright 2010-2016 Tormod Volden and Stefan Schmidt
This program is Free Software and has ABSOLUTELY NO WARRANTY
Please report bugs to http://sourceforge.net/p/dfu-util/tickets/

Cannot open Dfu device 1a86:df11
Opening Dfu capable USB device...
ID 1a86:df11
Run-time device Dfu version 011a
Claiming USB Dfu Interface...
Setting Alternate Setting #0 ...
Determining device status: state = dfuIDLE, status = 0
dfuIDLE, continuing
Dfu mode device Dfu version 011a
Device returned transfer size 1024
DfuSe interface name: "Internal Flash"
Limiting upload to end of memory segment, 45056 bytes
Upload [=====] 100% 45056 bytes
Upload done.
PS D:\下载\dfu-util-0.9-win64\dfu-util-0.9-win64> |
```

历史版本

| 日期        | 版本   | 变更内容 |
|-----------|------|------|
| 2026/8/27 | V1.0 | 初版发行 |

## 声明

本手册仅供参考。作者在不另行通知的情况下，可随时进行修改。